

OCR Document Extraction

The OCR feature lets users upload a document (PDF, image) on a public form and have the fields auto-filled by AI.

How It Works

User uploads file → POST /public/forms/:token/extract-document

↓

Backend converts file to base64

↓

Sends to Anthropic Claude (claude-sonnet-4-6) with:

- The image/PDF as a vision message
- A prompt listing fields and their allowed values

↓

Claude returns JSON: { fieldName: extractedValue }

↓

Frontend applies 4-tier normalization for SELECT/RADIO fields

↓

Form fields are pre-filled; user reviews and submits

Configuration

In the Form Builder → **OCR Upload** tab:

1. Toggle "Enable document upload"
2. Select which fields should be auto-filled

3. Save

This writes to `Form.settings` :

```
{
  "documents": {
    "enabled": true,
    "fieldIds": ["field-id-1", "field-id-2"]
  }
}
```

Backend Implementation

File: `backend/src/forms/forms.service.ts` → `extractDocument()`

Field Description Prompt

For SELECT/RADIO fields, the backend explicitly lists allowed values:

- "gender" (Gender) [MUST be one of: "male", "female", "other"]
- "full_name" (Full Name)
- "date_of_birth" (Date of Birth)

This reduces hallucination — Claude is instructed to return only the exact allowed values.

Claude API Call

```
const response = await this.anthropic.messages.create({
  model: 'claude-sonnet-4-6',
  max_tokens: 1024,
  messages: [{
    role: 'user',
    content: [
      {
        type: 'image',
        source: { type: 'base64', media_type: file.mimetype, data: base64Data }
      },
    ],
  }],
});
```

```

    {
      type: 'text',
      text: `Extract values for the following form fields...`
    }
  ]
}]
});

```

Supported File Types

Any MIME type Claude supports as vision input:

- `image/jpeg` , `image/png` , `image/webp` , `image/gif`
- `application/pdf`

File size limit: 10 MB (set in `main.ts` body parser config).

Frontend Option Normalization

Even though the backend prompt instructs Claude to use exact values, normalization is applied as a safety net:

```

const OPTION_TYPES = new Set(["RADIO", "SELECT", "DROPDOWN", "STATUS", "MULTI_SELECT"]);

// 4-tier match for each extracted value:
const match =
  opts.find(o => o.value === raw) // 1. Exact value
  || opts.find(o => o.value.toLowerCase() === lo) // 2. Case-insensitive value
  || opts.find(o => o.label.toLowerCase() === lo) // 3. Case-insensitive label
  || opts.find(o => // 4. Partial label
    o.label.toLowerCase().includes(lo) || lo.includes(o.label.toLowerCase())
  );

```

If no match is found, the raw extracted value is kept (the user will see it but it won't validate as a valid option).

Error Handling

Error	Cause	User Message
<code>ANTHROPIC_API_KEY</code> missing	Key not set in <code>backend/.env</code>	"Document extraction is not configured"
File too large	> 10 MB	"File too large"
Unsupported file type	Non-image/PDF	"Unsupported file type"
Claude API error	Rate limit, API down	The actual error message from Claude
No fields extracted	Document unreadable	"Could not extract data. Please fill manually."

Required Setup

1. Create an account at <https://console.anthropic.com>
2. Generate an API key
3. Add to `backend/.env` :

```
ANTHROPIC_API_KEY=sk-ant-api03-...
```

4. Restart the backend

The frontend also has `ANTHROPIC_API_KEY` in `.env.local` — this is for any direct frontend Claude calls (not OCR). The OCR feature exclusively uses the backend key.

Cost Estimate

OCR uses `claude-sonnet-4-6` with vision. Approximate cost per document scan:

- Input: ~1000 tokens (image + field prompt)
- Output: ~200 tokens (JSON response)
- At \$3/MTok input + \$15/MTok output $\approx$ ~\$0.006 per scan

For high-volume forms, consider caching extracted values or batching requests.